Singly Linked List –
Reverse List Using Recursion
and Time Complexity.

***Program*: –**

```
void reverseListUsingRecursion(Node **head)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
    else
    {
        Node *first = *head;
        Node *rest = first->next;
        if (rest == nullptr)
        {
            return;
        }
        reverseListUsingRecursion(&rest);
        first->next->next = first;
        first->next = nullptr;
        *head = rest;
    }
}
......
   case 16:
        {
            reverseListUsingRecursion(&head);
            break;
        }
```

Program Analysis:

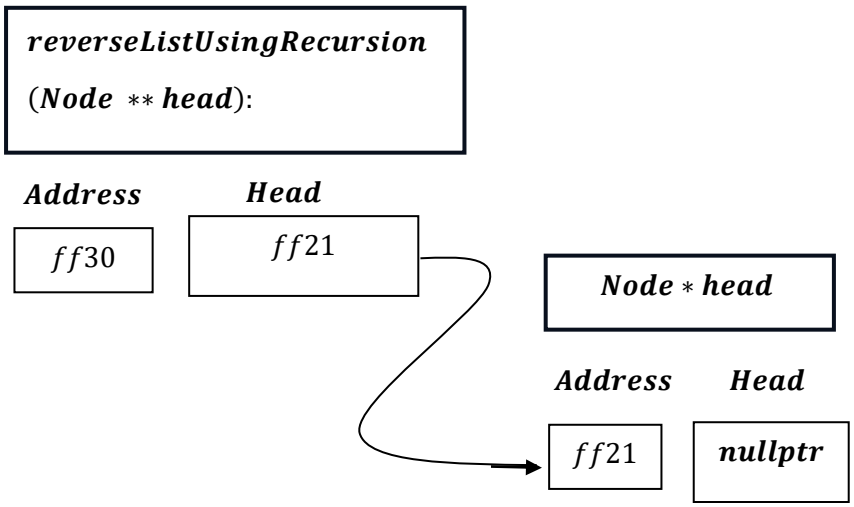
Now, as we call the function :

```
            reverseListUsingRecursion (&head);
```

Say, reverseListUsingRecursion(&head: ff21);

But our list is Empty, hence :

```
void reverseListUsingRecursion(Node **head) {…}
```

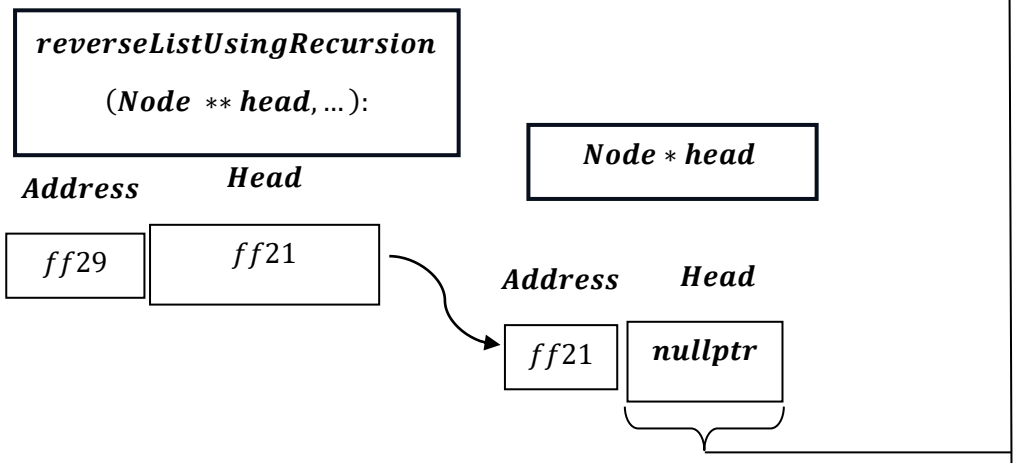


Now if Head pointer of Node structure points to nullptr then list is empty ,
i.e. Head carries value nullptr (not pointing to any valid memory location)

```
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
```

$if(*head == nullptr)$ *if it is true*

If true then Why true ?
*Ans: Though * head which is of Node ** head*
temporary pointer to pointer variable of the function
*, as * head is used , it dereference from Node ** head and*
*refer to Node * head global pointer to which Node ** head*
is pointing to and then its value is checked is
whether equal to nullptr or not ? This occurs during execution
and compilation of the reverseListUsingRecursion function.

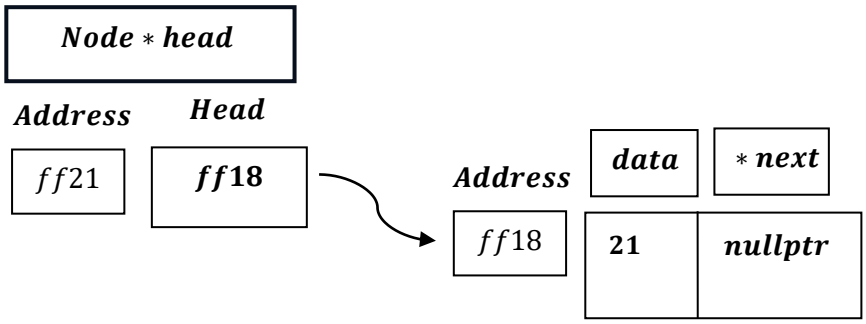


Then , it prints a message : `Error: List is empty.` and return.

Now, List is not Empty

1. *Single Node*

We are given with a Linked List:



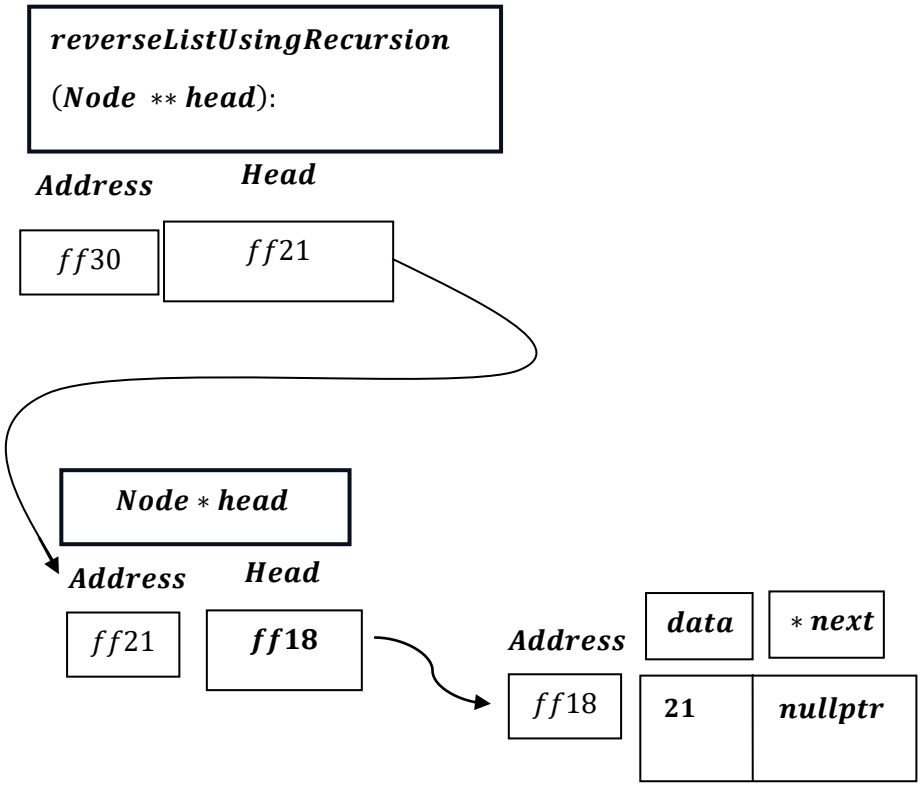
Next Function Call: ***reverseListUsingRecursion***(&***head***);

```
reverseListUsingRecursion (&head);
```

i.e. reverseListUsingRecursion(&***head***: ***ff21***);

Now:

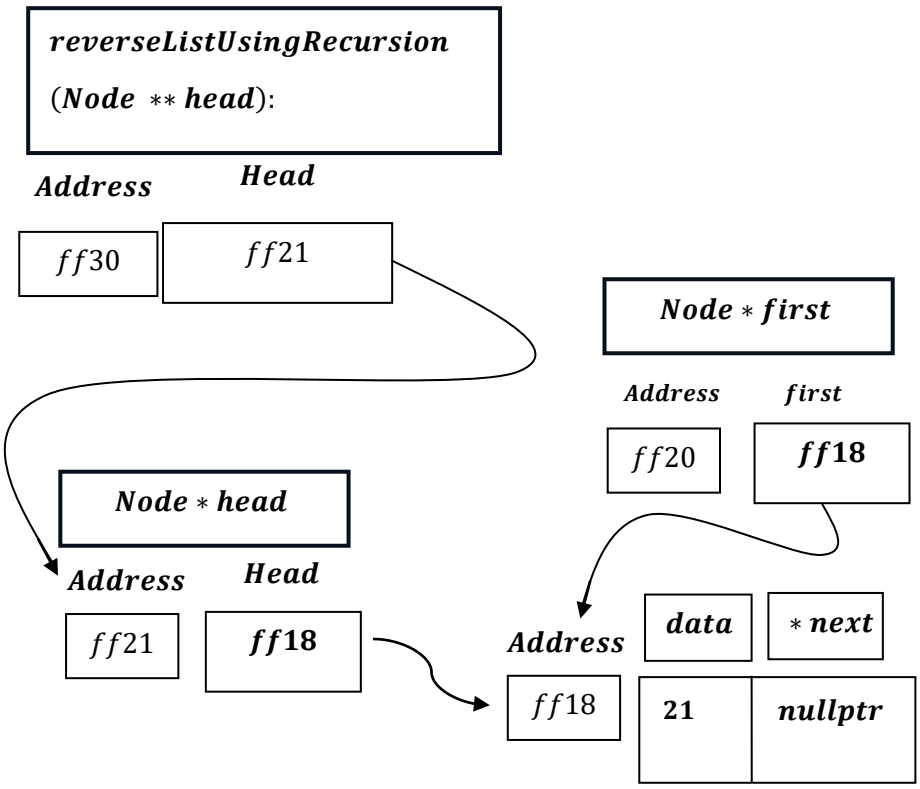
```
void reverseListUsingRecursion(Node **head) {…}
```



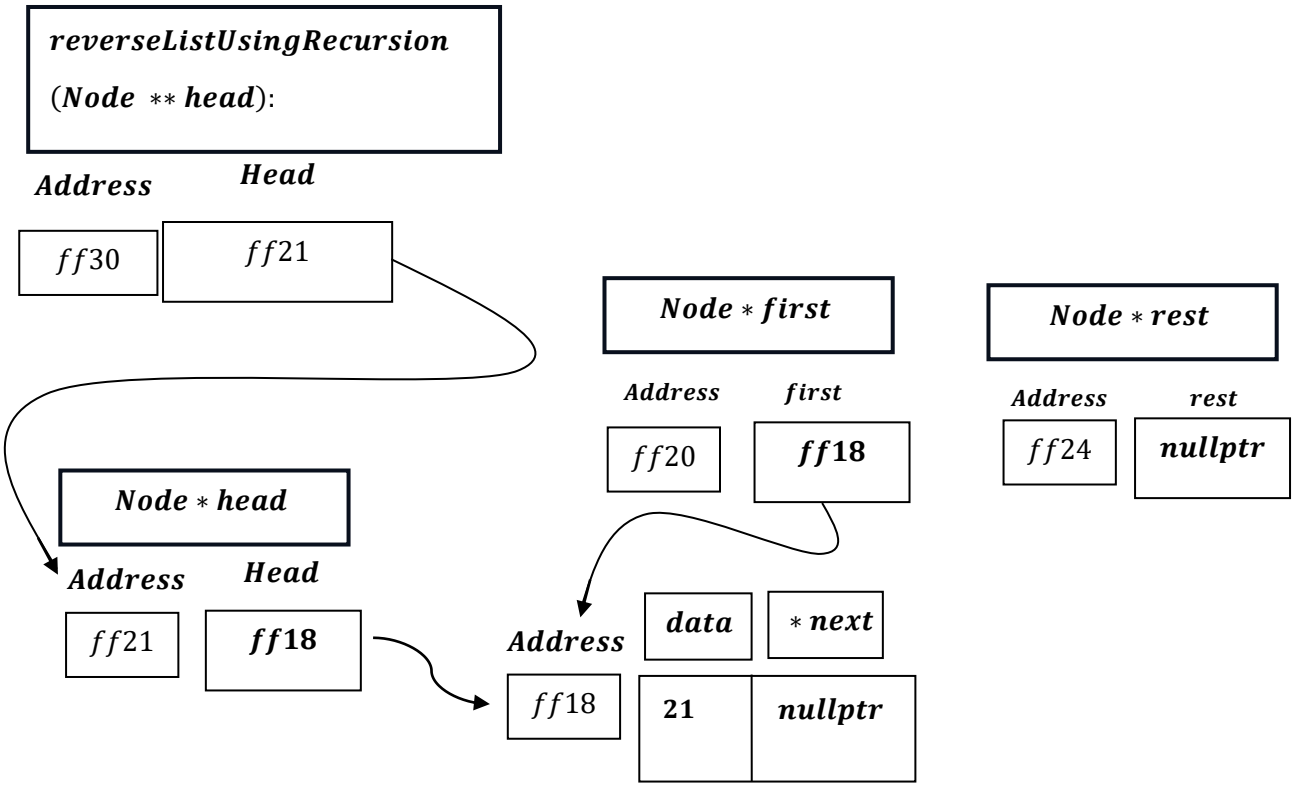
Now , come to else part:

```
    else
    {
        Node *first = *head;
        Node *rest = first->next;
        if (rest == nullptr)
        {
            return;
        }
        reverseListUsingRecursion(&rest);
        first->next->next = first;
        first->next = nullptr;
        *head = rest;
    }
}
```

$\boldsymbol{Therefore, Node * first = * head = ff18;}$



$\boldsymbol{Therefore, Node * rest = first \rightarrow next = nullptr;}$



Now, base case direct, as rest is nullptr, function returns .

```
        if (rest == nullptr)
        {
            return;
        }
```

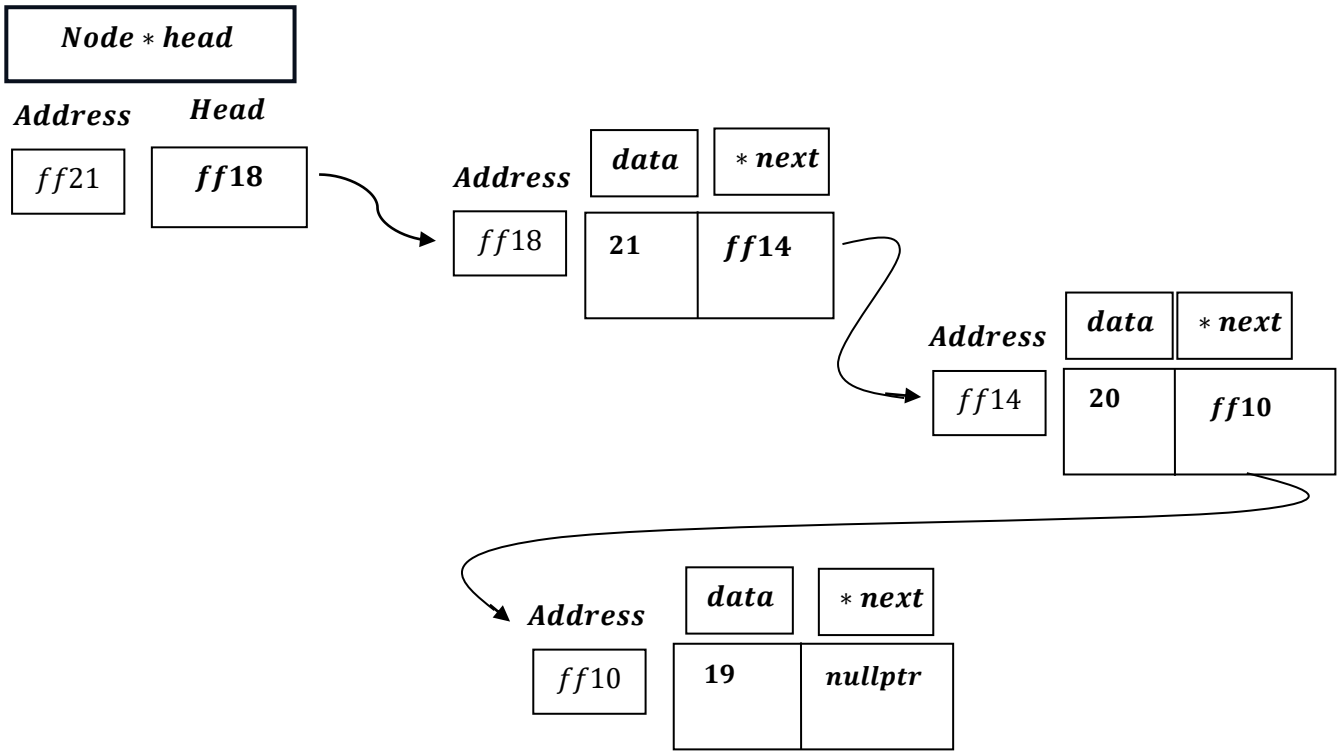
And as function returns immediately, there is no execution of :

```
        first->next->next = first;
        first->next = nullptr;
        *head = rest;
```

Hence making no changes and list remains unchanged.

2. Multiple Nodes

We are given with a Linked List:



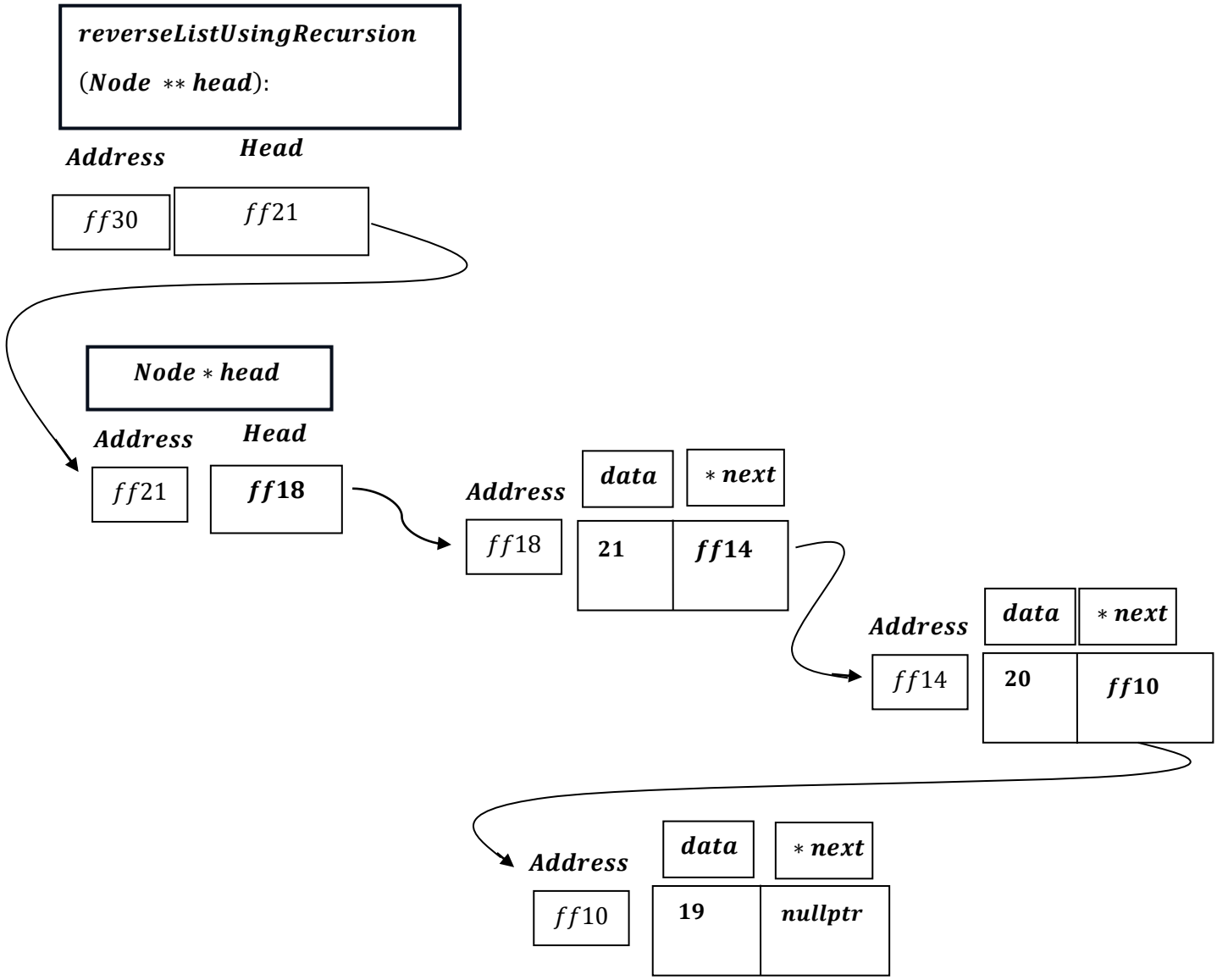
Next Function Call: ***reverseListUsingRecursion***(&***head***);

```
reverseListUsingRecursion (&head);
```

i.e. reverseListUsingRecursion(&***head***: ***ff21***);

Now:

```
void reverseListUsingRecursion(Node **head) {…}
```

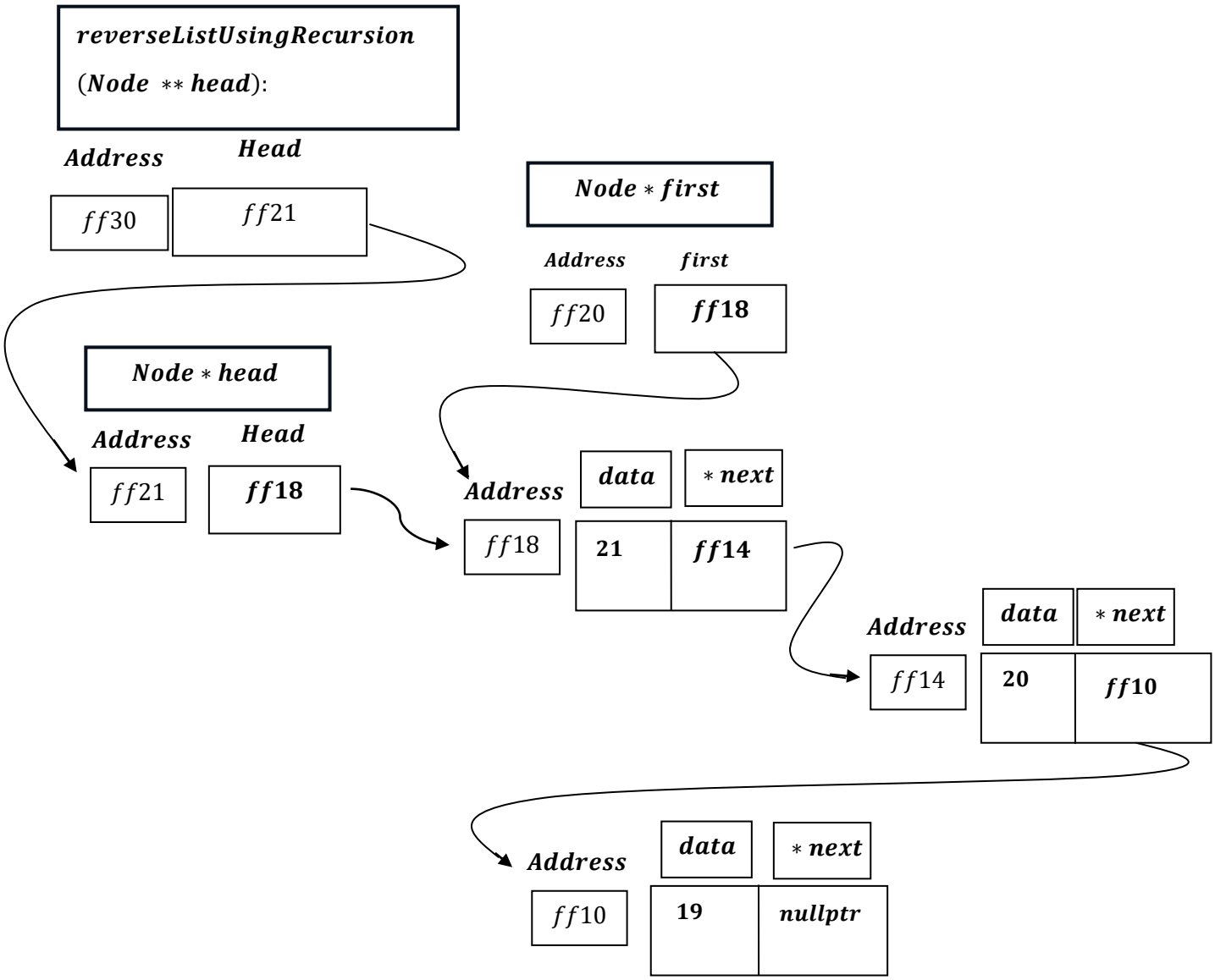


Now , come to else part:

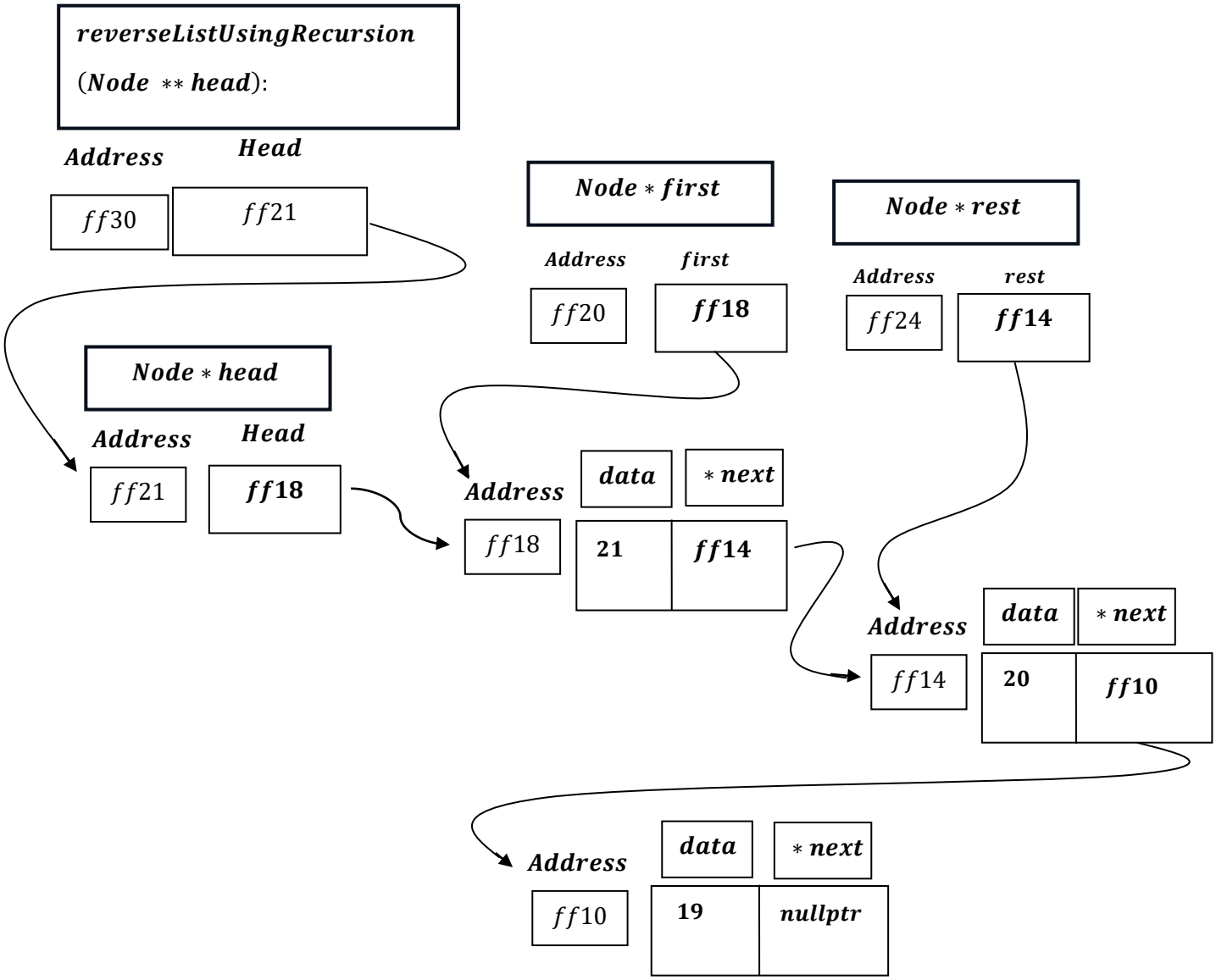
```
    else
    {
        Node *first = *head;
        Node *rest = first->next;
        if (rest == nullptr)
        {
            return;
        }
        reverseListUsingRecursion(&rest);
```

```
        first->next->next = first;
        first->next = nullptr;
        *head = rest;
    }
}
```

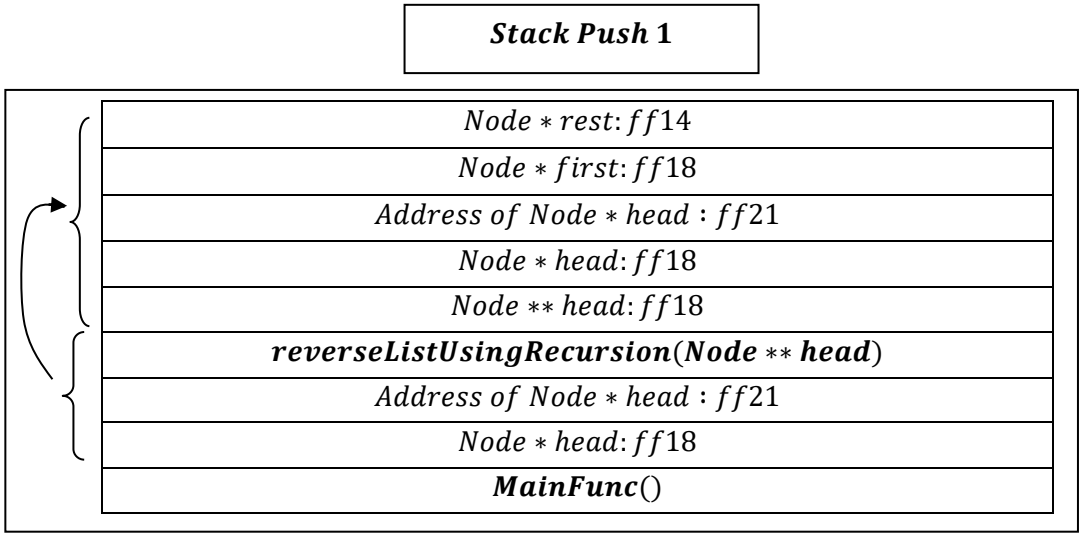
$Therefore, Node * first = * head = ff18;$



$Therefore, Node * rest = first \rightarrow next = ff14;$



Now, it check the Base Case that: if rest is `nullptr` , then the function returns void and recursion ends. As per now , the rest is not null hence , before recursive call our stack is:

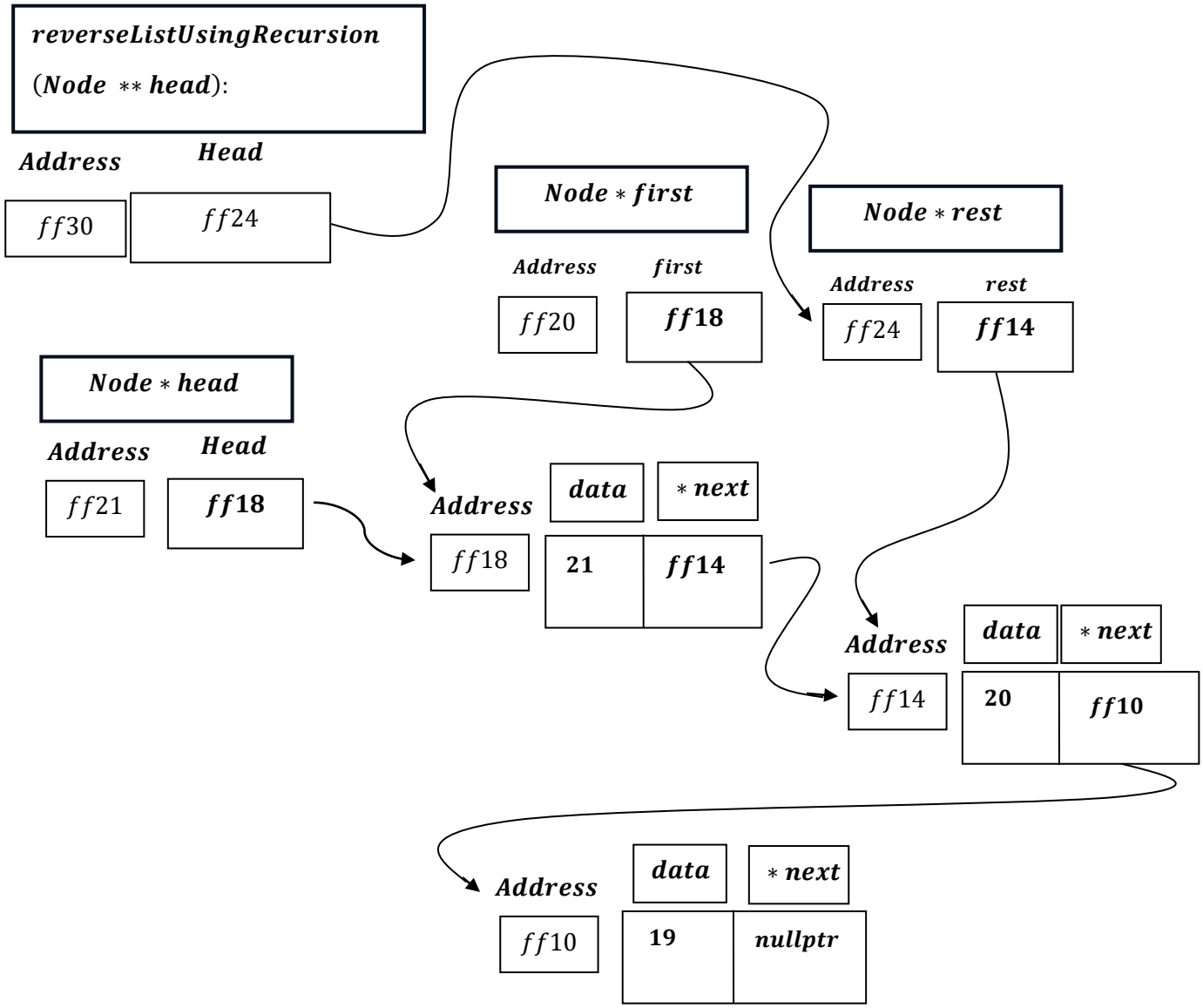


Now recursive call is made:

```
        reverseListUsingRecursion(&rest);
```

***i.e.*, *recursive call* : *reverseListUsingRecursion*(&*rest*: *ff24*);**

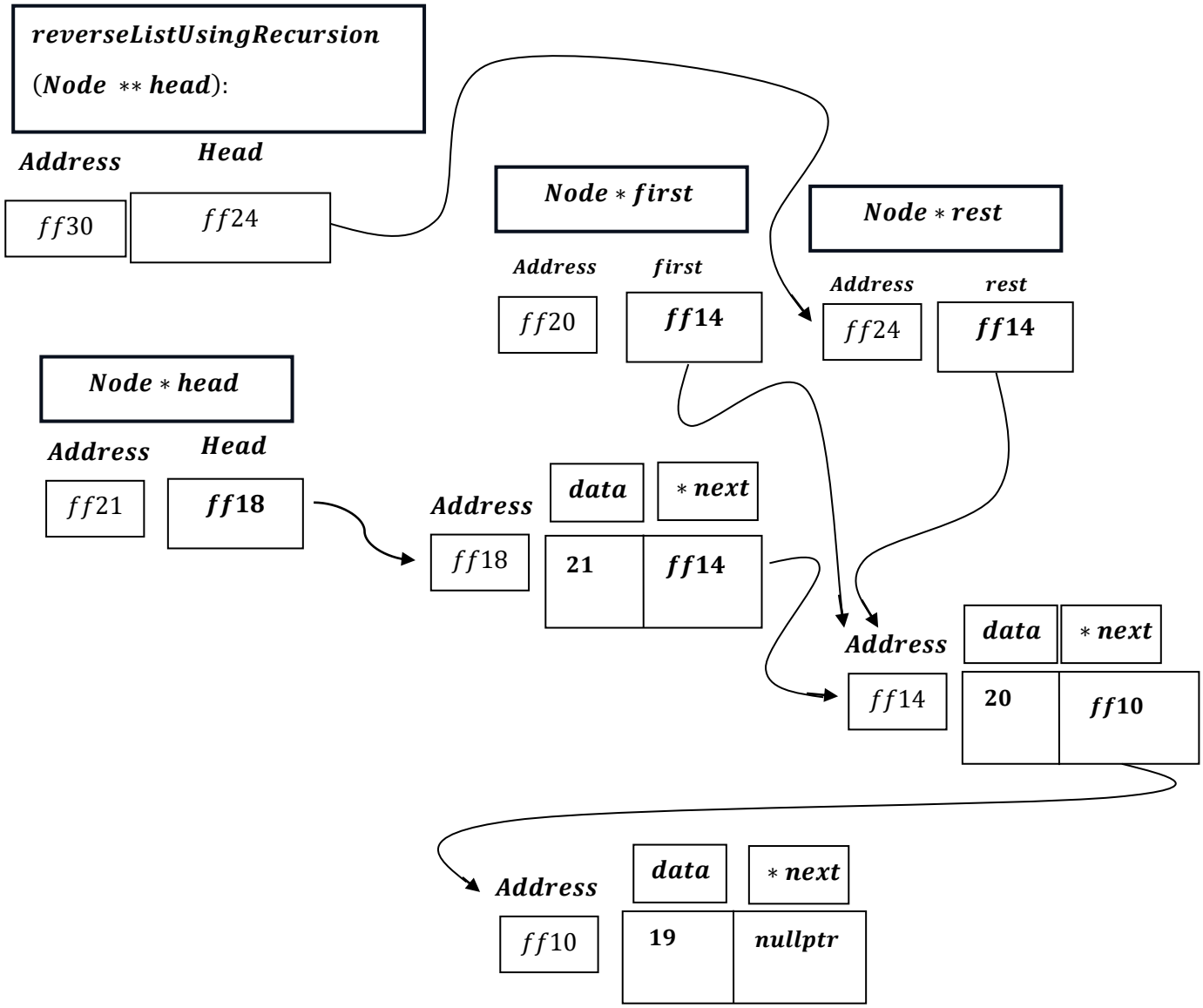
Hence ,



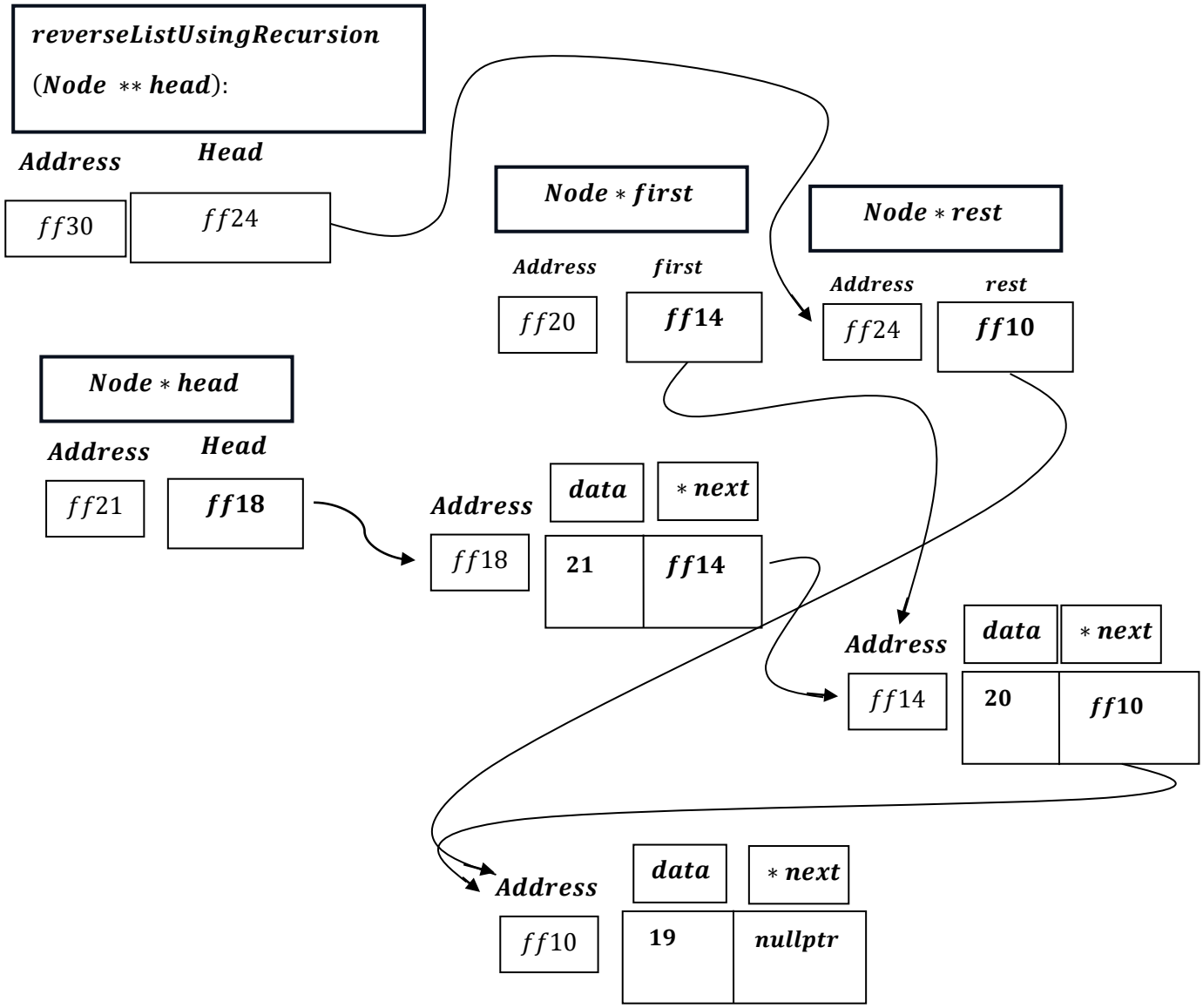
Now,

$$Node * first = * head = \ ff14;$$

Now, first pointer is assigned to the address of the value (Address) hold by rest local pointer variable of Node Structure.



$Node * rest = first \rightarrow next = ff10;$



Hence on 2nd stack push we get:

Stack Push 2

$Node * rest: ff10$
$Address\ of\ Node * rest : ff24$
$Node * first: ff14$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$Node ** head: ff24$
$\boldsymbol{reverseListUsingRecursion(Node ** head)}$
$Node * rest: ff14$
$Node * first: ff18$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$Node ** head: ff18$
$\boldsymbol{reverseListUsingRecursion(Node ** head)}$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$\boldsymbol{MainFunc()}$

$\boldsymbol{Now\ recursive\ call\ is\ made}$:

```
        reverseListUsingRecursion(&rest);
```

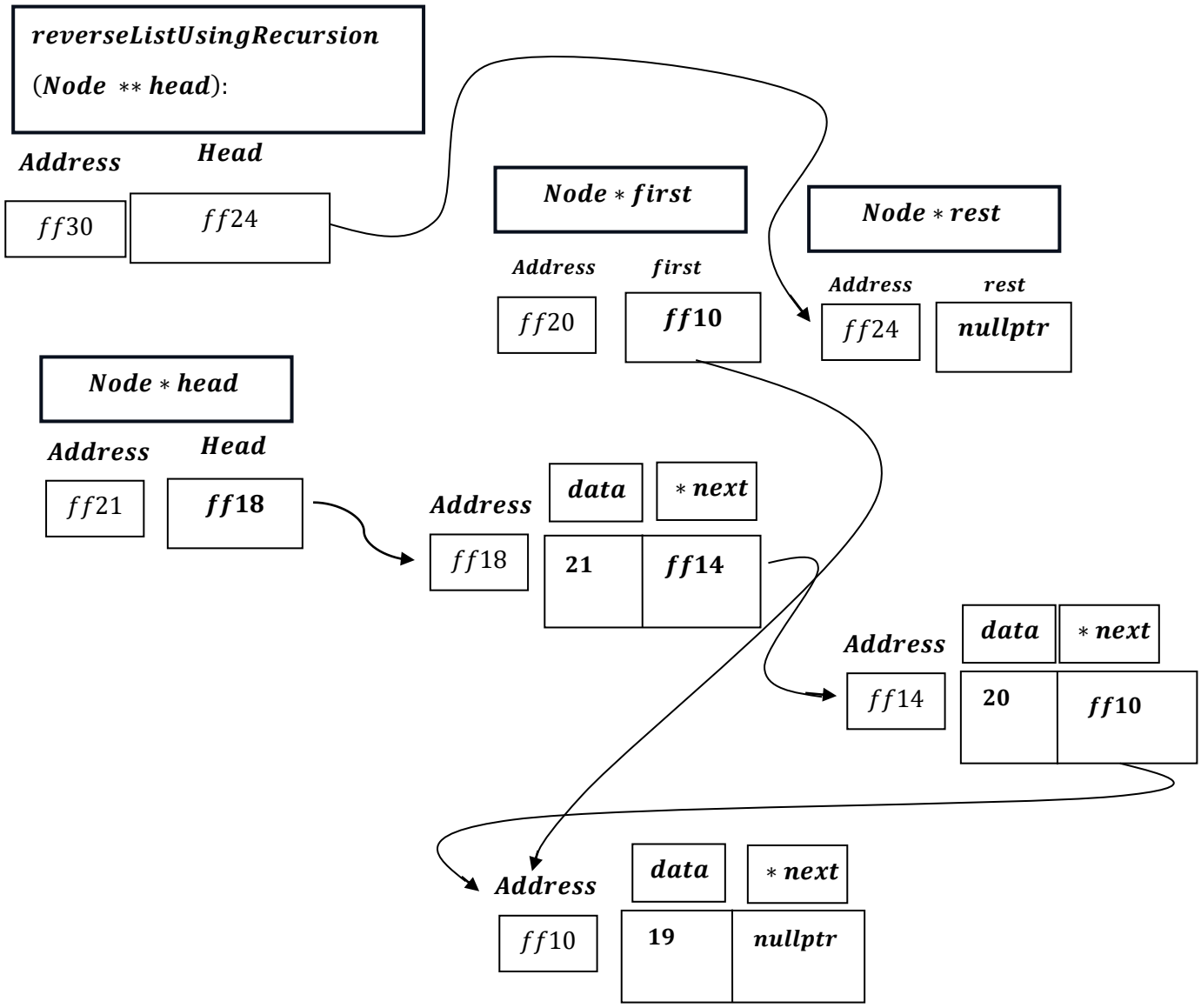
$\boldsymbol{i.e., recursive\ call : reverseListUsingRecursion(\&rest: ff24)}$;

$\boldsymbol{Now}$,

$\boldsymbol{Node * first = * head = \ ff10}$;

$\boldsymbol{Node * rest = \ first \rightarrow next = nullptr}$;

$\boldsymbol{pictorially\ represented\ as}$:


reverseListUsingRecursion
(Node ** head):
Address
Head
ff30
ff24
Node * first
Address
first
ff20
ff10
Node * rest
Address
rest
ff24
nullptr
Node * head
Address
Head
ff21
ff18
Address
data
* next
ff18
21
ff14
Address
data
* next
ff14
20
ff10
Address
data
* next
ff10
19
nullptr

And ,

$Stack\ Push\ 3$

$Node * rest: nullptr$
$Address\ of\ Node * rest : ff24$
$Node * first: ff10$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$Node ** head: ff24$
$\boldsymbol{reverseListUsingRecursion(Node ** head)}$
$Node * rest: ff10$
$Address\ of\ Node * rest : ff24$
$Node * first: ff14$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$Node ** head: ff24$
$\boldsymbol{reverseListUsingRecursion(Node ** head)}$
$Node * rest: ff14$
$Node * first: ff18$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$Node ** head: ff18$
$\boldsymbol{reverseListUsingRecursion(Node ** head)}$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$\boldsymbol{MainFunc()}$

***Now , as recursion had over , pop will begin, Stack Pop 1*:**

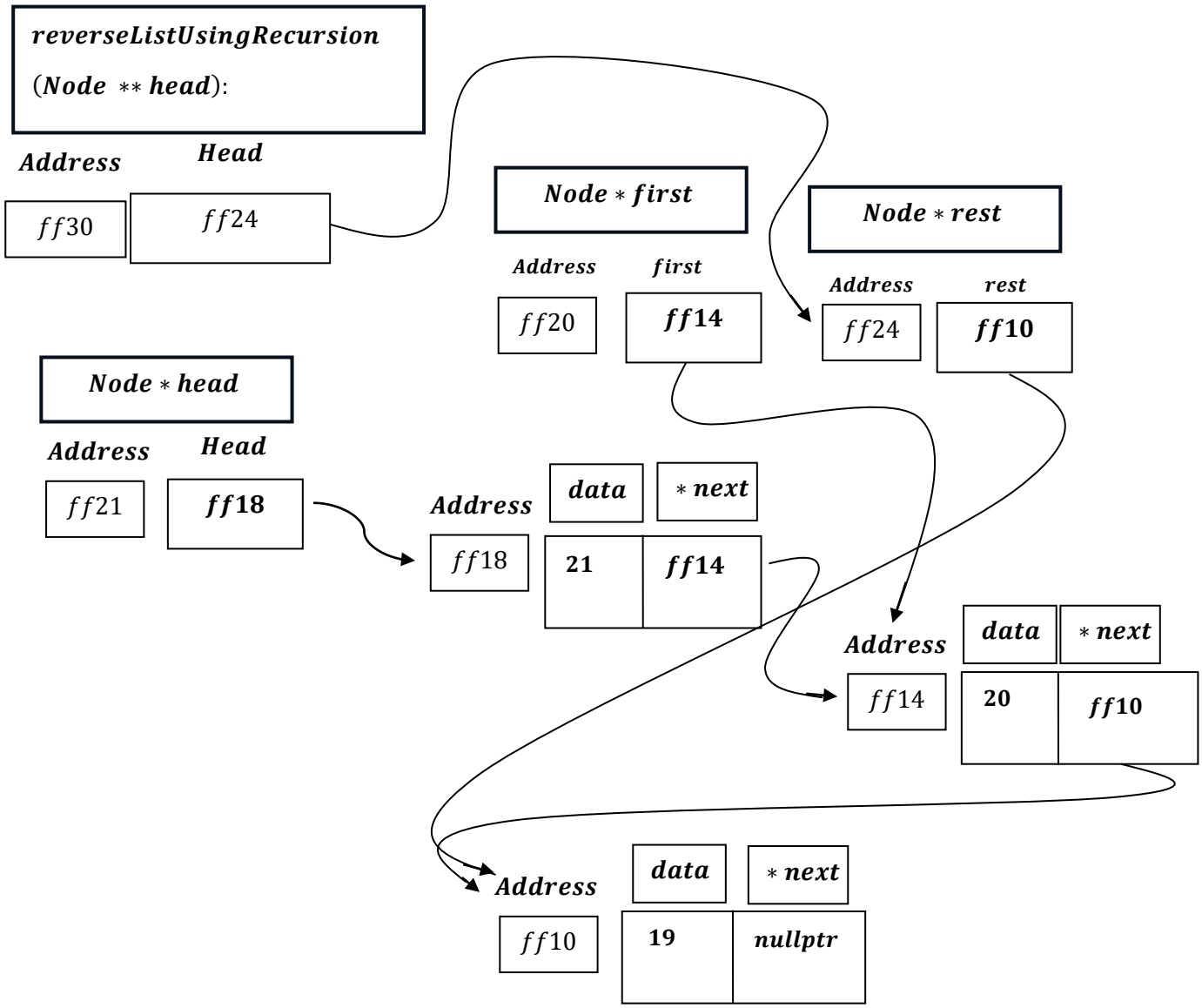
$Stack\ Pop\ 1$

$Node * rest: nullptr$
$Address\ of\ Node * rest : ff24$
$Node * first: ff10$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$Node ** head: ff24$
$\boldsymbol{reverseListUsingRecursion(Node ** head)}$

Now, above section of memory will be deallocated and remaining stack is:

$Node * rest: ff10$
$Address\ of\ Node * rest : ff24$
$Node * first: ff14$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$Node ** head: ff24$
$\boldsymbol{reverseListUsingRecursion(Node ** head)}$
$Node * rest: ff14$
$Node * first: ff18$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$Node ** head: ff18$
$\boldsymbol{reverseListUsingRecursion(Node ** head)}$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$\boldsymbol{MainFunc()}$

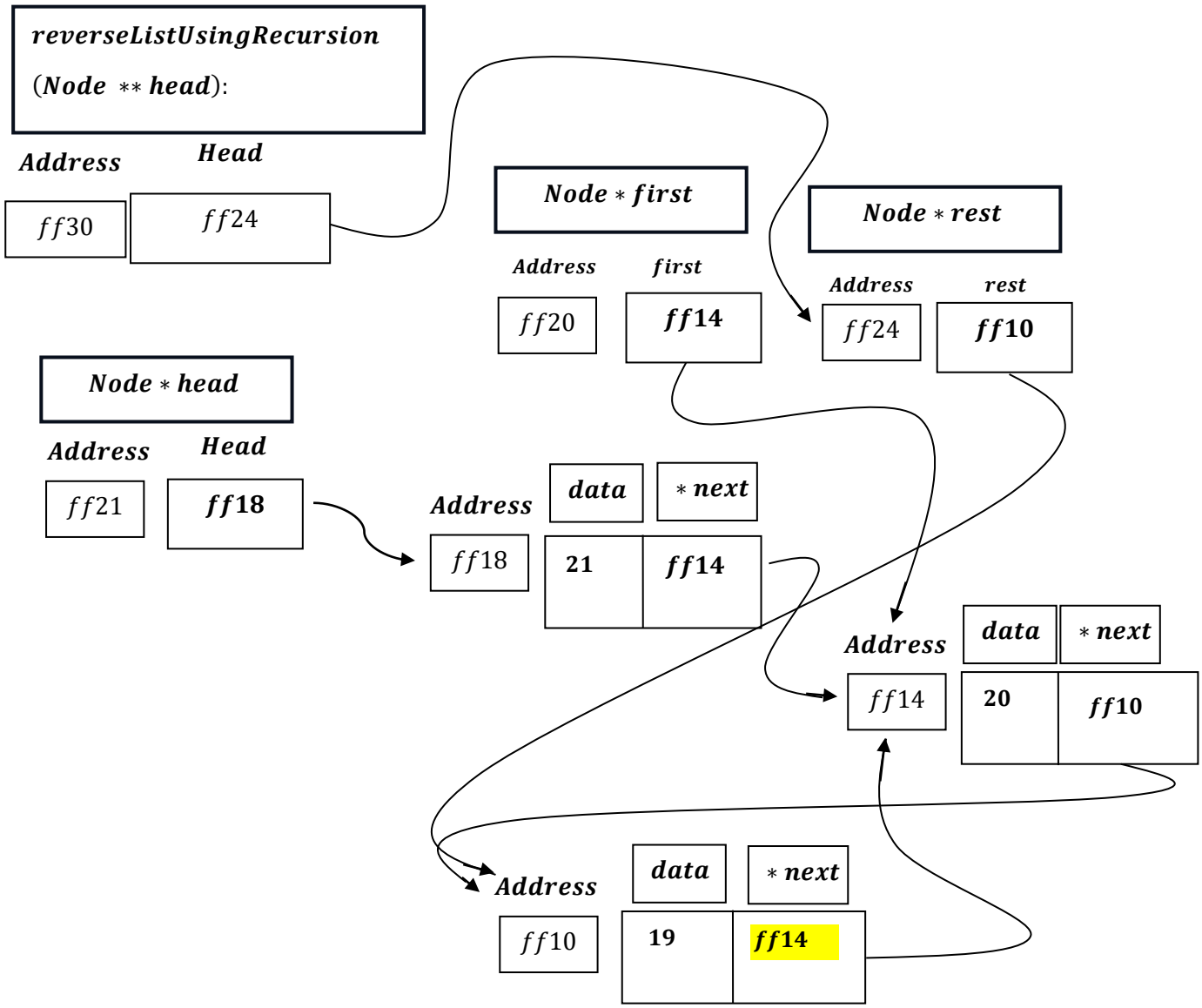
Hence, now $Node * rest$ is: $ff10$ and $Node * first$: $ff14$, which can be shown pictorially below:



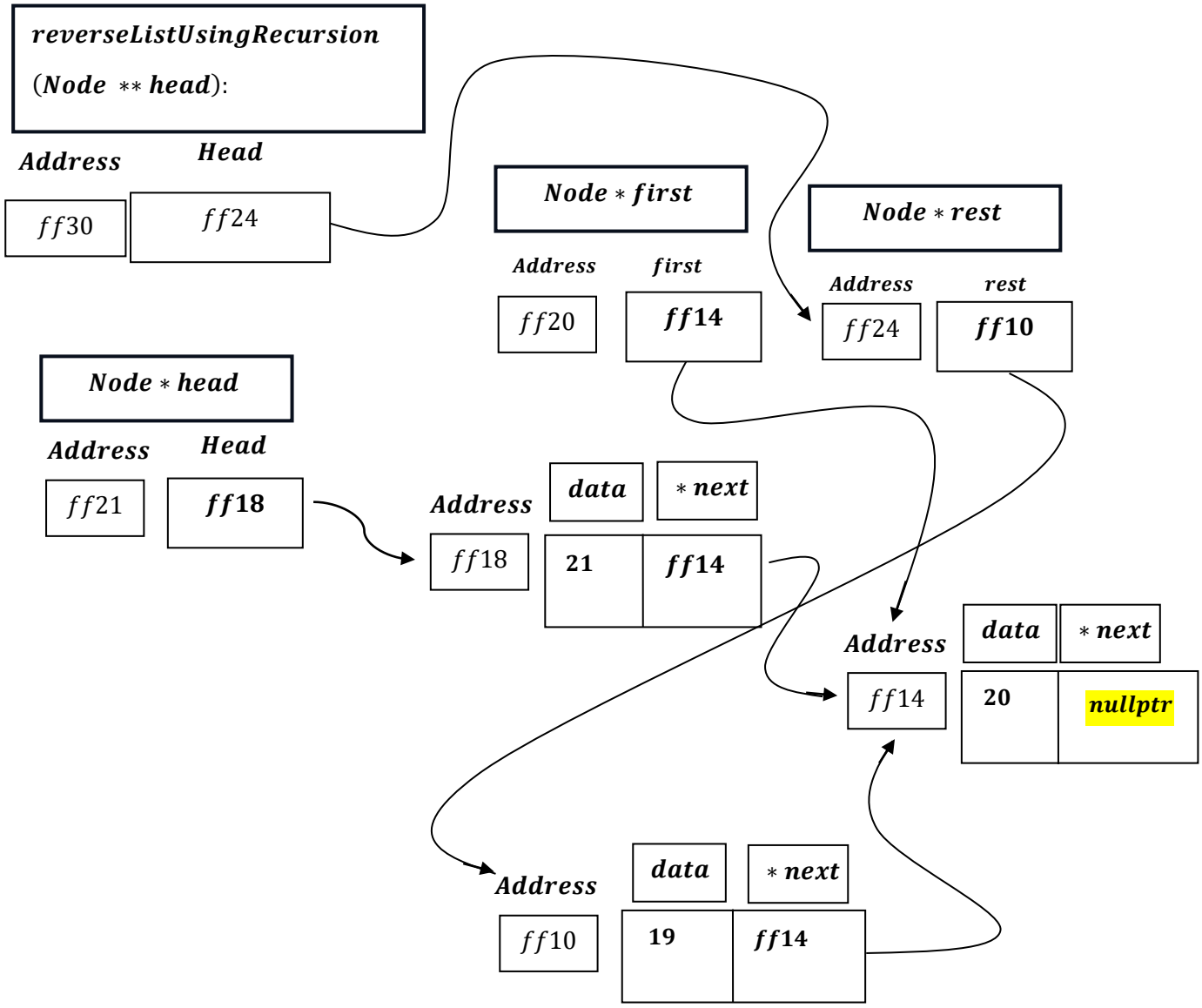
And now,

```
        first->next->next = first;
        first->next = nullptr;
        *head = rest;
```

$first \rightarrow next \rightarrow next = first = ff14$;



$first \rightarrow next = nullptr;$



$*head = rest = ff10;$

As, the assignment is done after the recursion call, therefore the change we observe in stack i.e. rest which the head is pointed to will become $ff10$.

$Node * rest: ff10$
$Address\ of\ Node * rest : ff24$
$Node * first: ff14$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$Node ** head: ff24$
$\boldsymbol{reverseListUsingRecursion(Node ** head)}$
$Node * rest: ff10$
$Node * first: ff18$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$Node ** head: ff18$
$\boldsymbol{reverseListUsingRecursion(Node ** head)}$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$\boldsymbol{MainFunc()}$

And the above setion of the memory pops out:

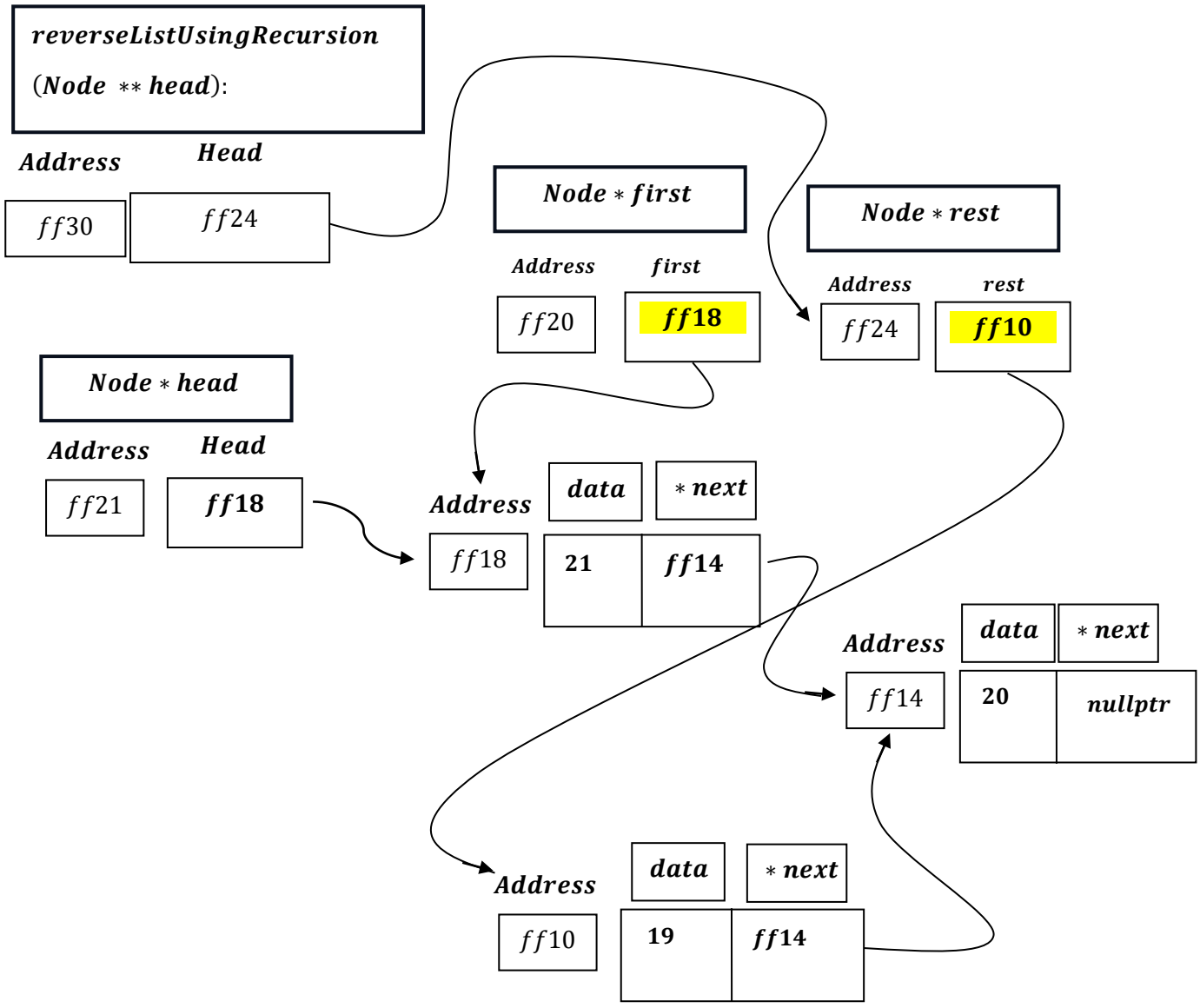
***Stack Pop* 2**

$Node * rest: ff10$
$Address\ of\ Node * rest : ff24$
$Node * first: ff14$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$Node ** head: ff24$
$\boldsymbol{reverseListUsingRecursion(Node ** head)}$

Now, above section of memory will be deallocated and remaining stack is:

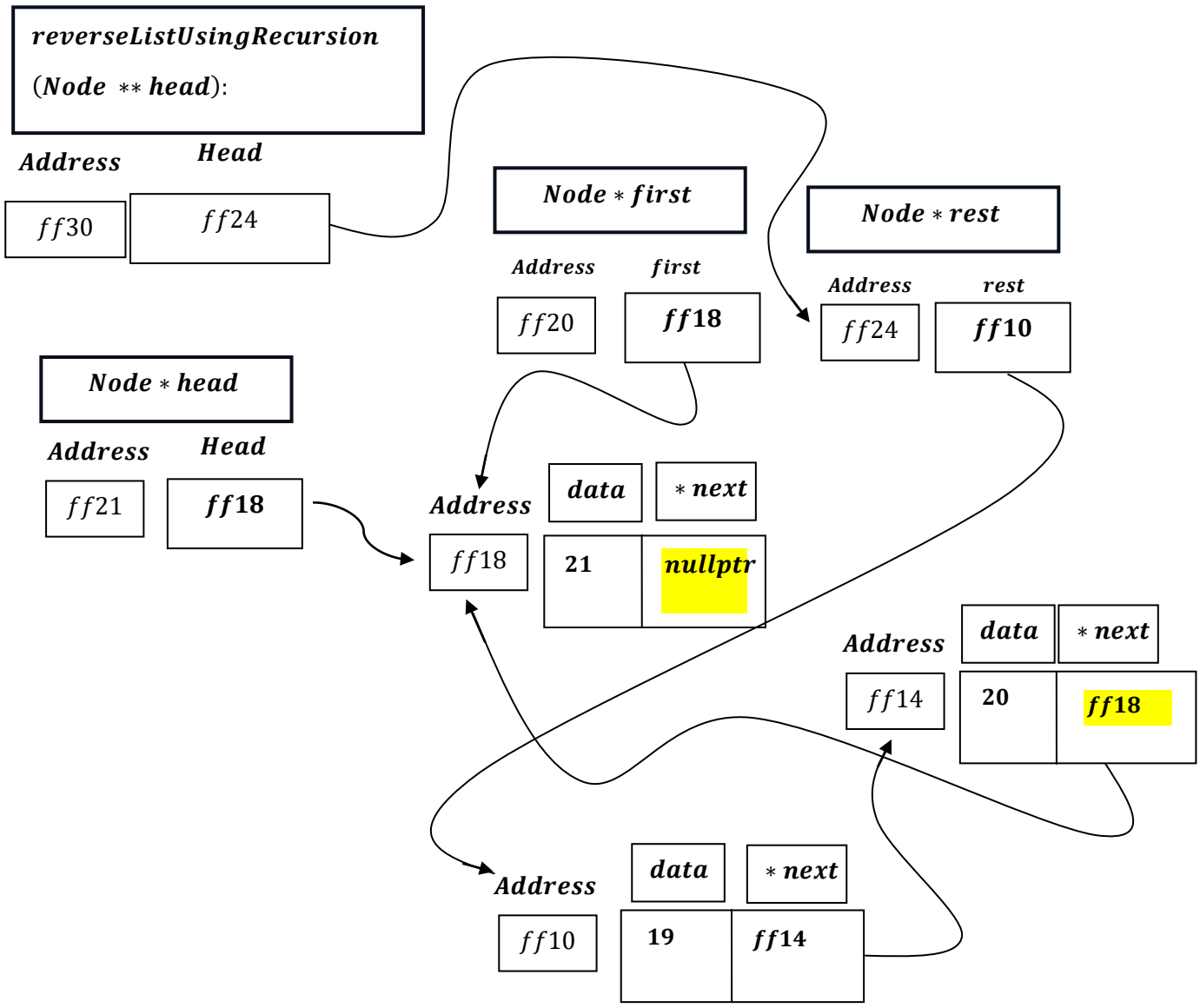
$Node * rest: ff10$
$Node * first: ff18$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$Node ** head: ff18$
$\boldsymbol{reverseListUsingRecursion(Node ** head)}$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$\boldsymbol{MainFunc()}$

Hence , now Node * rest is: ff10 and Node * first: ff18 :



$first \rightarrow next \rightarrow next = first = ff18$;

$first \rightarrow next = nullptr$;



$* head = rest = ff10$;

*As, the assignment is done after the recursion call, therefore the change we observe that again after pop Node ** head will point to Node * head, hence Node * head will get assigned to ff10 i.e. the rest value .*

$Node * rest: ff10$
$Node * first: ff18$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$Node ** head: ff18$
$\boldsymbol{reverseListUsingRecursion(Node ** head)}$
$Address\ of\ Node * head : ff21$
$Node * head: ff10$
$\boldsymbol{MainFunc()}$

And the above setion of the memory pops out:

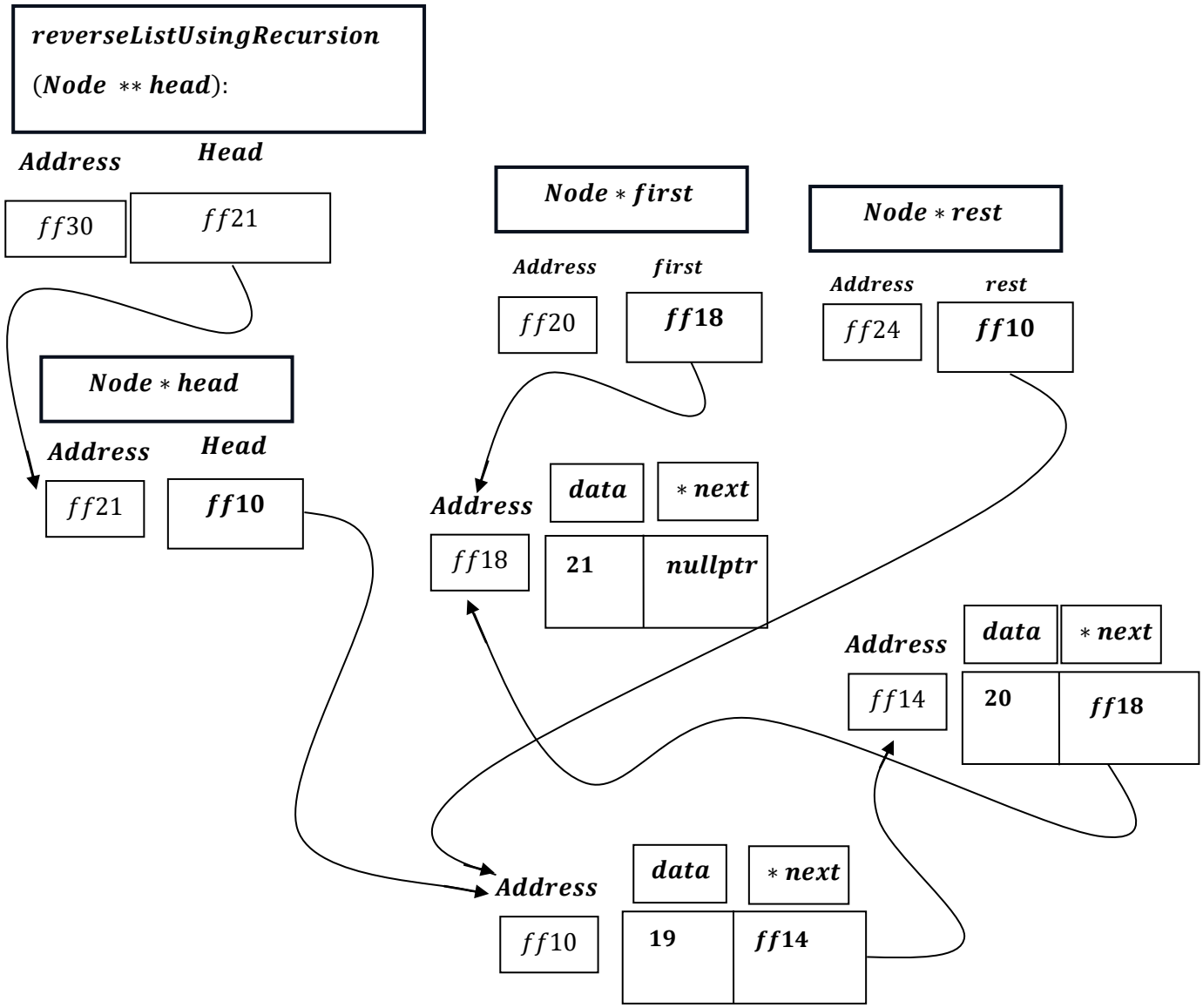
Stack Pop 3

$Node * rest: ff10$
$Node * first: ff18$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$Node ** head: ff18$
$\boldsymbol{reverseListUsingRecursion(Node ** head)}$

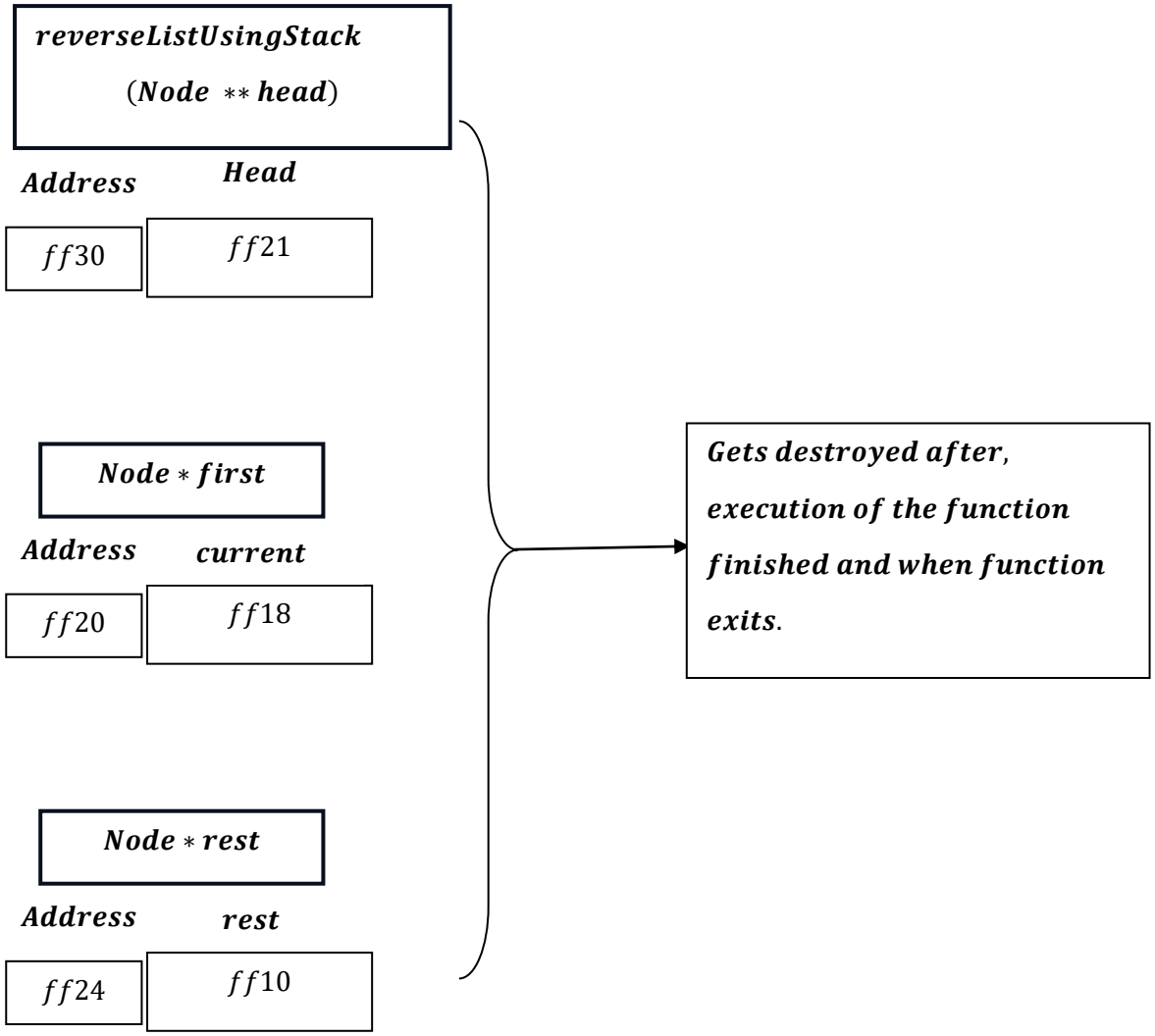
Now, above section of memory will be deallocated and remaining stack is:

$Address\ of\ Node * head : ff21$
$Node * head: ff10$
$\boldsymbol{MainFunc()}$

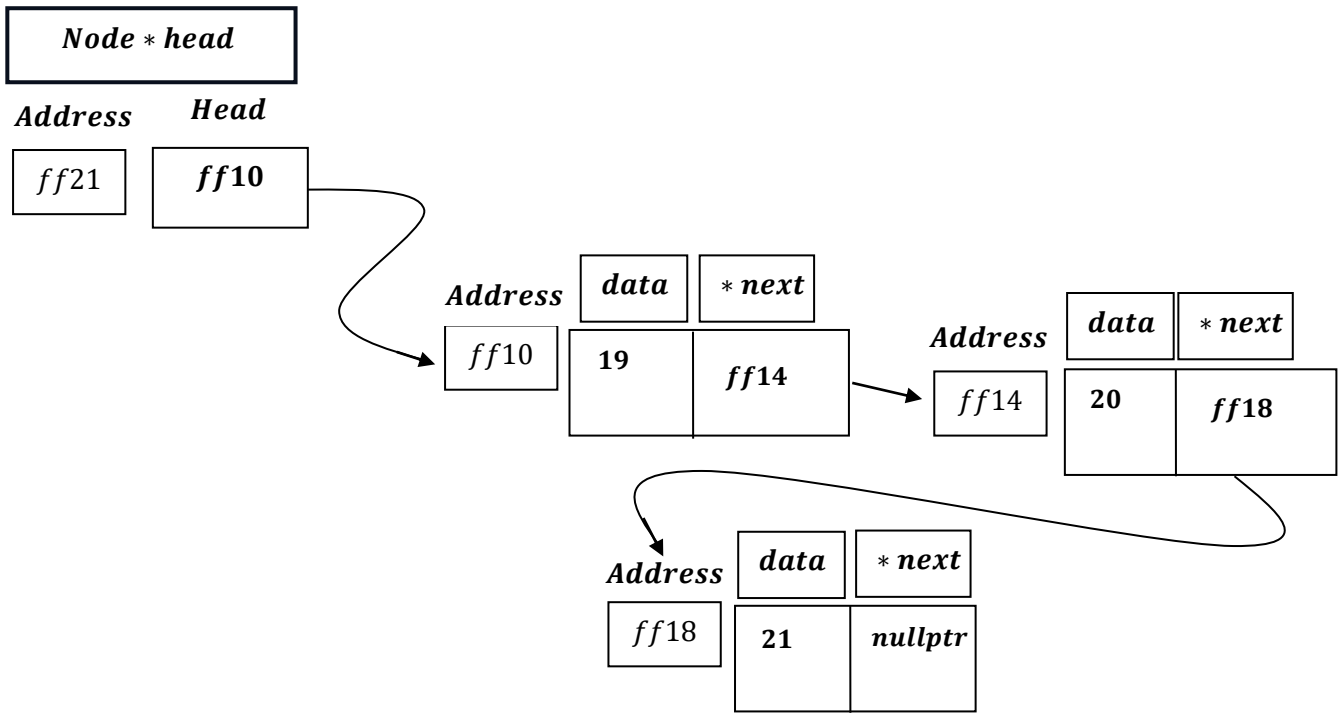
And ,



Now as the function ends , all the temporary pointer variables gets destroyed:



And we are left with:



The `reverseListUsingRecursion`, step by step recursion:

1. ***Base Case***: ***Check if the List is Empty***

   – ***The function first checks if the list is empty by checking if `* head` is `nullptr`***.

   – ***If the list is empty, it prints an error message and exits the function***.

2. ***Initialize Pointers***:

   – ***`first` is set to point to the first node in the list*** (`* head`).
   – ***`rest` is set to point to the second node in the list*** (`first → next`).

3. *Base Case*: *Check if the Rest pointer is Empty*

– *The function checks if the rest of the list* (`*rest*`) *is empty*.

– *If* `*rest*` *is* `*nullptr*`, *it means there is only one node in the list, and no further action is needed. The function returns*.

4. *Recursive Call*:

– *The function calls itself recursively with the address of* `*rest*` (`&*rest*`).
– *This recursive call will continue until it reaches the end of the list*.

5. *Reversing the Links*:

– *After the recursive call returns, the function reverses the link between the first node and the rest of the list*.

– `*first* → *next* → *next* = *first*;` *sets the* `*next*` *pointer of the second node* (*which is now the last node in the reversed list*)*to point back to the first node*.

– `*first* → *next* = *nullptr*;` *sets the* `*next*` *pointer of the first node to* `*nullptr*`, *indicating that it is now the last node in the reversed list*.

6. *Update the Head Pointer*

– *Finally, the function updates the head pointer to point to the new head of the reversed list* (`*rest*`).

Time Complexity of
Reverse List Using Recursion

```
void reverseListUsingRecursion(Node **head)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
    else
    {
        Node *first = *head;
        Node *rest = first->next;
        if (rest == nullptr)
        {
            return;
        }
        reverseListUsingRecursion(&rest);
        first->next->next = first;
        first->next = nullptr;
        *head = rest;
    }
}
```

$O(1)$

$O(1)$

$O(1)$

$O(1)$

$T(n) = n$ *at worst case.*

$O(1)$

$O(1)$

$O(1)$

Best Case

1. *When List is Empty.*

→ *If the list is empty (`* head` is `nullptr`), print an error message and return from the function.*

This takes $O(1)$ *time or* $\Omega(1)$ *time* .

2. When There Exists Single Node List.

→ If the list is empty (` ∗ head` is `nullptr`), print an error message and return from the function. Though List is not empty here hence Checking takes $O(1)$ constant time.

−As it moves to else part:

→ Node ∗ first =∗ head ; takes constant amount of time i.e. $O(1)$.

→ Node ∗ rest = first → next; takes constant amount of time i.e. $O(1)$. Note, Node ∗ rest = first → next = `nullptr` .

→ Now if(rest == nullptr) which is true and function returns and exists , takes constant amount of time i.e. $O(1)$.

Hence, total time complexity: $O(1) + O(1) + O(1) + O(1) = O(1)$.

Hence, this takes $O(1)$time or $\Omega(1)$time .

Worst Case(When List is not Empty)

→ If the list is empty (` ∗ head` is `nullptr`), print an error message and return from the function. Though List is not empty here hence Checking takes $O(1)$ constant time.

−As it moves to else part:

→ Node ∗ first =∗ head ; takes constant amount of time i.e. $O(1)$.

→ Node ∗ rest = first → next; takes constant amount of time i.e. $O(1)$.

→ reverseListUsingRecursion(&rest); recursion runs i.e. push and pop operation occurs `n` times results $O(n)$ time complexity.

→ As rest pointer has value now `nullptr` , the base case reached , hence returns

$and\ it\ \ takes\ constant\ amount\ of\ time\ i.e.\ O(1).$

$\rightarrow first{-}{>}\ next{-}{>}\ next\ =\ first;\ \ takes\ constant\ amount\ of\ time\ i.e.\ O(1).$

$\rightarrow\ \ first{-}{>}\ next\ =\ nullptr;\ \ takes\ constant\ amount\ of\ time\ i.e.\ O(1).$

$\rightarrow *\ head\ =\ rest;\ takes\ constant\ amount\ of\ time\ i.e.\ O(1).$

<u>Note</u> **:**

$first \rightarrow next \rightarrow next\ =\ first;$ ***This is a simple pointer assignment. It takes a constant amount of time to change the `next` pointer of a node.***

`$first \rightarrow next\ =\ nullptr;$`: ***Similarly, this is another pointer assignment. Setting the `next` pointer of `first` to `nullptr` takes constant time.***

`$*\ head\ =\ rest;$`: ***This is a pointer assignment as well. Updating the head pointer to point to `rest` takes constant time.***

Hence, total time complexity takes: $O(1) + O(1) + O(1) + O(n) + O(1) + O(1) + O(1) + O(1)$ $= O(n)$.

Hence, Worst Case time complexity is : $O(n)$.

Average Case

We can say, if List is Empty, we get our best case scenario, and when list is not empty, we get our worst case scenario. But though best case and worst case scenarios are present as, always we can determine list get traversed from beginning to end, hence we can say average case is our worst case here i.e. always the entire list is traversed i.e. $O(n) = \Theta(n)$.

$This\ means\ the\ execution\ time\ is\ solely\ dependent\ on\ the\ length\ of\ the\ list.$
$If\ the\ list\ is\ empty, the\ execution\ time\ is\ constant\ (best\ case).$
$If\ the\ list\ is\ not\ empty, the\ execution\ time\ is\ proportional\ to\ the$
$length\ of\ the\ list\ (worst\ case). Since\ the\ recursion\ is\ predetermined,$
$there's\ no\ element\ of\ randomness\ or\ varying\ input\ data\ to\ consider.$
$Therefore, the\ concept\ of\ an\ average\ case, where\ we\ average\ the\ execution$
$time\ across\ different\ input\ possibilities, doesn't\ strictly\ apply\ here.$
$Hence\ Reverse\ List\ Using\ Recursion\ doesn't\ have\ a\ true\ average\ case.$

$Note\ , by\ relation: \Omega \leq \Theta \leq O\ , we\ get:$

$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$

$\Rightarrow c_1 \times 1 \leq N \leq c_2 \times N$

$Singly\ Linked\ List$	$Cases$	$Time\ Complexity$
$Reverse\ List\ Using\ Recursion$	$1. Best\ Case$	$\Omega(1)$
	$2. Worst\ Case$	$O(n)$
	$3. Average\ Case\ (= Worst\ Case)$	$\Theta(n)$